

优极限

“极限教育，挑战极限”

www.yjxxt.com

极限教育，挑战极限。优极限是一个让 95% 的学生年薪过 18 万的岗前培训公司，让我们的学员具备优秀的互联网技术和职业素养，勇攀高薪，挑战极限。公司位于上海浦东，拥有两大校区，共万余平。累计培训学员超 3 万名。我们的训练营就业平均月薪 19000，最高年薪 50 万。

核心理念：让学员学会学习，拥有解决问题的能力，拿到高薪职场的钥匙。

项目驱动式团队协作、一对一服务、前瞻性思维、教练式培养模型-培养你成为就业明星。首创的老学员项目联盟给学员充分的项目、技术支撑，利用优极限平台这根杠杆，不断挑战极限，勇攀高薪，开挂人生。

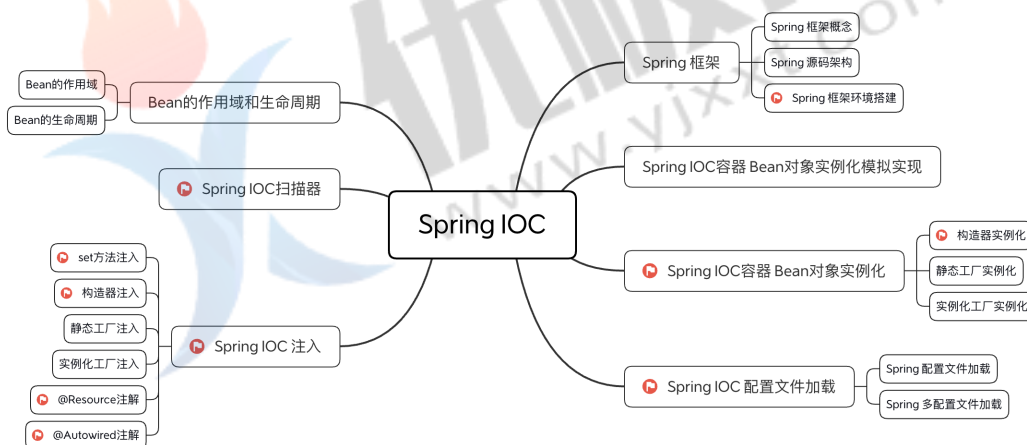
扫码关注优极限微信公众号：

（获取最新技术相关资讯及更多源码笔记）



Spring IOC

主要内容



Spring 框架

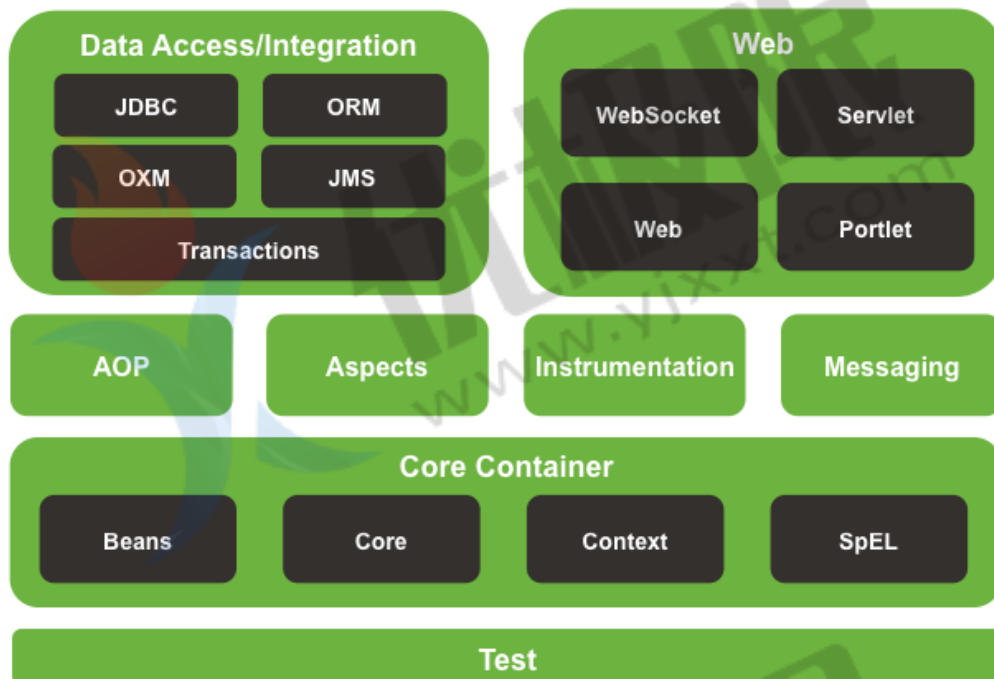
Spring 框架概念

Spring 是众多开源java项目中的一员，基于分层的javaEE应用一站式轻量级开源框架，主要核心是IOC（控制反转/依赖注入）与 AOP（面向切面）两大技术，实现项目在开发过程中的轻松解耦，提高项目的开发效率。

在项目中引入 Spring 立即可以带来下面的好处 降低组件之间的耦合度,实现软件各层之间的解耦。可以使用容器提供的众多服务，如：事务管理服务、消息服务等等。当我们使用容器管理事务时，开发人员就不再需要手工控制事务,也不需处理复杂的事务传播。容器提供单例模式支持，开发人员不再需要自己编写实现代码。容器提供了AOP技术，利用它很容易实现如权限拦截、运行期监控等功能。



Spring Framework Runtime



Spring 源码架构

Spring 总共大约有20个模块，由1300多个不同的文件构成。而这些组件被分别整合在核心容器（Core Container）、Aop（Aspect Oriented Programming）和设备支持（Instrumentation）、数据访问及集成（Data Access/Integration）、Web、报文发送（Messaging）、测试6个模块集合中。

1. 核心容器：Spring-beans 和 Spring-core 模块是 Spring 框架的核心模块，包含控制反转（Inversion of Control, IoC）和依赖注入（Dependency Injection, DI），核心容器提供 Spring 框架的基本功能。核心容器的主要组件是 BeanFactory，工厂模式的实现。BeanFactory 使用控制反转（IOC）思想将应用程序的配置和依赖性规范与实际的应用程序代码分开。

Spring 上下文 Spring Context：Spring 上下文是一个配置文件，向 Spring 框架提供上下文信息。Spring 上下文包括企业服务，例如 JNDI、EJB、电子邮件、国际化、校验和调度功能。

Spring-Expression 模块是统一表达式语言（unified EL）的扩展模块，可以查询、管理运行中的对象，同时也方便的可以调用对象方法、操作数组、集合等。它的语法类似于传统 EL，但提供了额外的功能，最出色的要数函数调用和简单字符串的模板函数。

2. Spring-AOP：Spring-aop 是 Spring 的另一个核心模块，在 Spring 中，他是以 JVM 的动态代理技术为基础，然后设计出了一系列的 Aop 横切实现，比如前置通知、返回通知、异常通知等。通过其配置管理特性，Spring AOP 模块直接将面向切面的编程功能集成到了 Spring 框架中。所以，可以很容易地使 Spring 框架管理的任何对象支持 AOP。
3. Spring Data Access(数据访问)：由 Spring-jdbc、Spring-tx、Spring-orm、Spring-jms 和 Spring-oxm 5 个模块组成。Spring-jdbc 模块是 Spring 提供的 JDBC 抽象框架的主要实现模块，用于简化 Spring JDBC。

Spring-tx 模块是 Spring JDBC 事务控制实现模块。使用 Spring 框架，它对事务做了很好的封装，通过它的 Aop 配置，可以灵活的配置在任何一层。

Spring-Orm 模块是 ORM 框架支持模块，主要集成 hibernate, Java Persistence API (JPA) 和 Java Data Objects (JDO) 用于资源管理、数据访问对象(DAO)的实现和事务策略。

Spring-Jms 模块（Java Messaging Service）能够发送和接受信息。

Spring-Oxm 模块主要提供一个抽象层以支撑 OXM（OXM 是 Object-to-XML-Mapping 的缩写，它是一个 O/M-mapper，将 java 对象映射成 XML 数据，或者将 XML 数据映射成 java 对象），例如：JAXB, Castor, XMLBeans, JiBX 和 XStream 等。

4. Web 模块：由 Spring-web、Spring-webmvc、Spring-websocket 和 Spring-webmvc-portlet 4 个模块组成，Web 上下文模块建立在应用程序上下文模块之上，为基于 Web 的应用程序提供了上下文。Web 模块还简化了处理多部分请求以及将请求参数绑定到域对象的工作。

5. 报文发送：即 Spring-messaging 模块。

Spring-messaging 是 Spring 4 新加入的一个模块，主要职责是为 Spring 框架集成一些基础的报文传送应用。

6. 单元测试：即 Spring-test 模块。Spring-test 模块主要为测试提供支持

Spring 框架环境搭建

环境要求

JDK 版本：

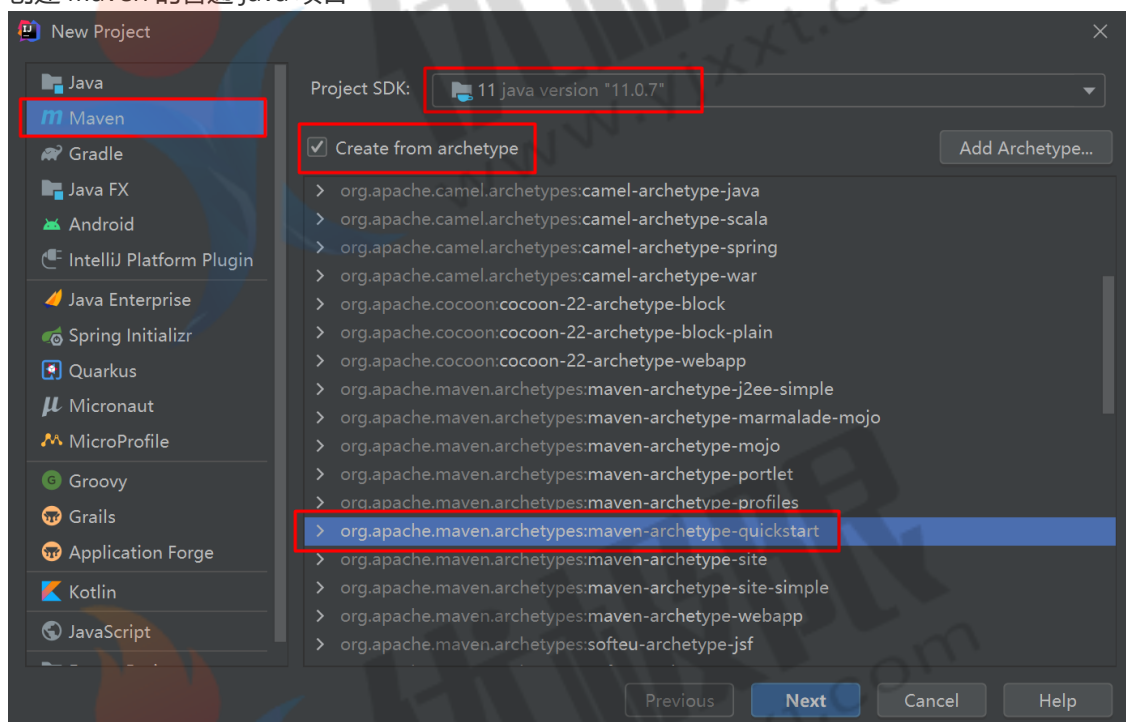
JDK 1.7 及以上版本

Spring 版本：

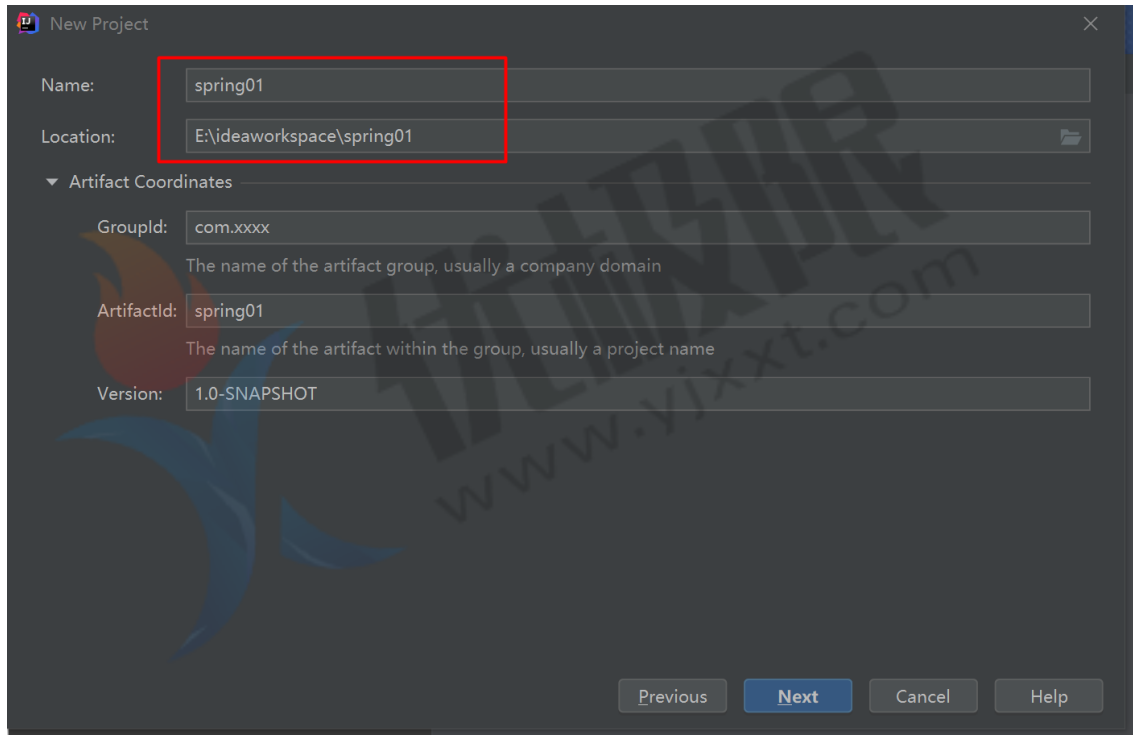
Spring 5.x 版本

新建 Maven 项目

1. 创建 Maven 的普通 Java 项目



2. 设置项目的坐标、名称、工作空间



New Project

Name:

Location:

▼ Artifact Coordinates

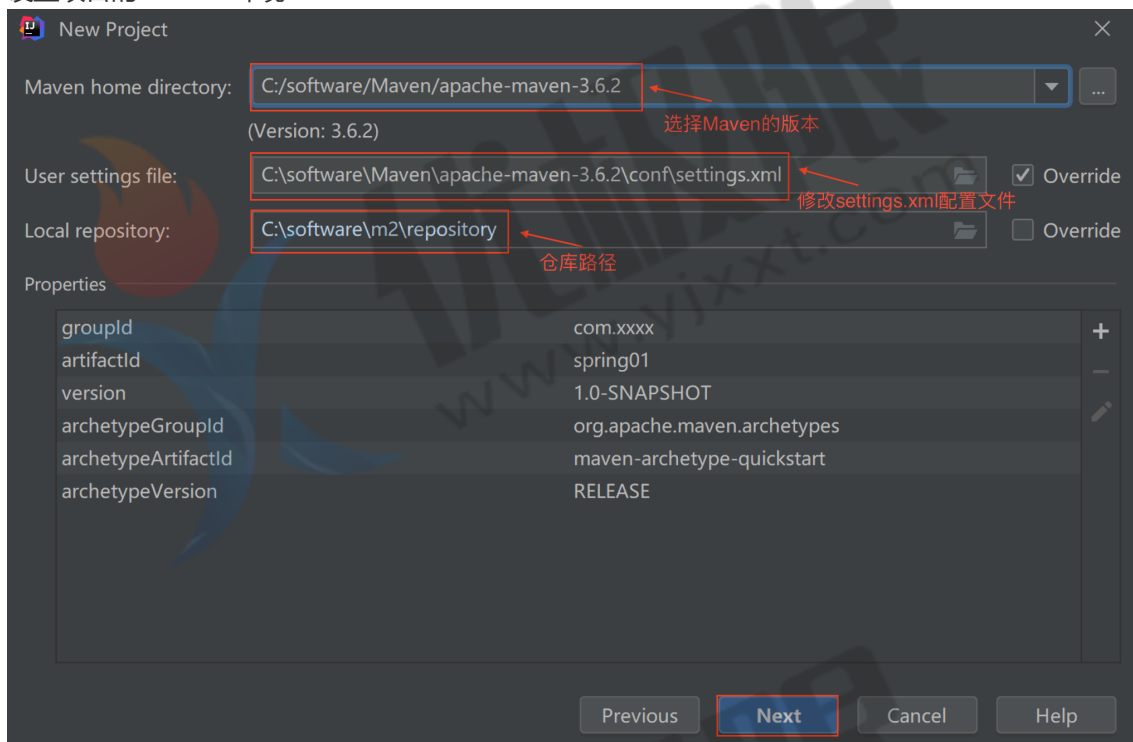
GroupId:
The name of the artifact group, usually a company domain

ArtifactId:
The name of the artifact within the group, usually a project name

Version:

Previous Next Cancel Help

3. 设置项目的 Maven 环境



New Project

Maven home directory: (Version: 3.6.2) 选择Maven的版本

User settings file: 修改settings.xml配置文件 ☒ Override

Local repository: 仓库路径 ☐ Override

Properties

groupId	com.xxxx
artifactId	spring01
version	1.0-SNAPSHOT
archetypeGroupId	org.apache.maven.archetypes
archetypeArtifactId	maven-archetype-quickstart
archetypeVersion	RELEASE

Previous Next Cancel Help

调整项目环境

1. 修改 JDK 版本

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
</properties>
```

2. 修改单元测试 JUnit 版本

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
```

3. build标签中的pluginManagement标签

```
<!--删除build标签中的pluginManagement标签-->
<build>
</build>
```

添加 Spring 框架的依赖坐标

Maven仓库: <https://mvnrepository.com/>

```
<!-- 添加Spring框架的核心依赖 -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.2.4.RELEASE</version>
</dependency>
```

编写 Bean 对象

```
package com.xxxx.service;

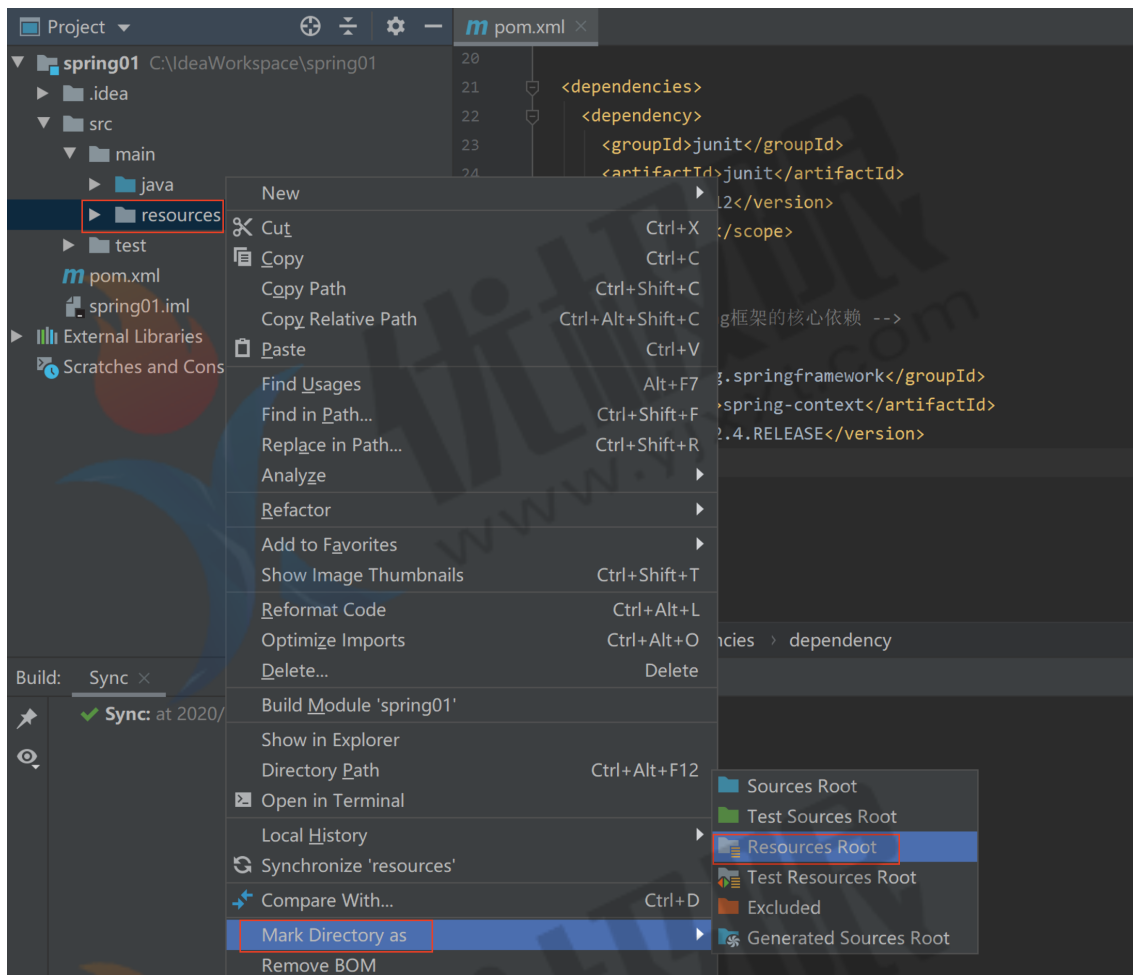
public class UserService {

    public void test(){
        System.out.println("Hello Spring!");
    }

}
```

添加Spring 配置文件

1. 在项目的src下创建文件夹 resources (Alt+insert)
2. 将 resources 标记为资源目录



- 在 src\main\resources 目录下新建 spring.xml 文件，并拷贝官网文档提供的模板内容到 xml 中。
配置 bean 到 xml 中，把对应 bean 纳入到 Spring 容器来管理
spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!--
        xmlns 即 xml namespace xml使用的命名空间
        xmlns:xsi 即xml schema instance xml 遵守的具体规范
        xsi:schemaLocation 本文档xml遵守的规范 官方指定
    -->
    <bean id="userService" class="com.xxxx.service.UserService"></bean>

</beans>
```

- 在 spring.xml 中配置 Bean 对象

```
<!--
    id: bean对象的id, 唯一标识。一般是Bean对象的名称的首字母小写
    class: bean对象的类路径
-->
<bean id="userService" class="com.xxxx.service.UserService"></bean>
```

加载配置文件，获取实例化对象

```
package com.xxxx;

import com.xxxx.service.UserService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class App {
    public static void main(String[] args) {
        // 获取Spring上下文环境（加载配置文件）
        ApplicationContext ac = new
ClassPathXmlApplicationContext("spring.xml");
        // 通过getBean方法得到Spring容器中实例化好的Bean对象（实例化Bean对象）
        // userService代表的是配置文件中bean标签的id属性值
        UserService userService = (UserService) ac.getBean("userService");
        // 调用方法（使用实例化对象）
        userService.test();
    }
}
```

Spring IOC 容器 Bean 对象实例化模拟

思路:

1. 定义Bean 工厂接口，提供获取bean方法
2. 定义Bean工厂接口实现类，解析配置文件，实例化Bean对象
3. 实现获取Bean方法

定义 Bean 属性对象

```
package com.xxxx.spring;

/**
 * bean对象
 * 用来接收配置文件中bean标签的id与class属性值
 */
public class MyBean {

    private String id; // bean对象的id属性值
    private String clazz; // bean对象的类路径

    public MyBean() {
    }

    public MyBean(String id, String clazz) {
        this.id = id;
        this.clazz = clazz;
    }

    public String getId() {
        return id;
    }
}
```

```

    }

    public void setId(String id) {
        this.id = id;
    }

    public String getClazz() {
        return clazz;
    }

    public void setClazz(String clazz) {
        this.clazz = clazz;
    }
}

```

添加 dom4j 坐标依赖

```

<!-- dom4j -->
<dependency>
    <groupId>dom4j</groupId>
    <artifactId>dom4j</artifactId>
    <version>1.6.1</version>
</dependency>
<!-- XPath -->
<dependency>
    <groupId>jaxen</groupId>
    <artifactId>jaxen</artifactId>
    <version>1.1.6</version>
</dependency>

```

准备自定义配置文件

spring.xml

```

<?xml version="1.0" encoding="utf-8" ?>
<beans>
    <bean id="userService" class="com.xxxx.service.UserService"></bean>
    <bean id="accountService" class="com.xxxx.service.AccountService"></bean>
</beans>

```

定义 Bean 工厂接口

```

package com.xxxx.spring;

/**
 * Bean 工厂接口定义
 */
public interface MyFactory {
    // 通过id值获取对象
    public Object getBean(String id);
}

```

定义 Bean 接口的实现类

```

package com.xxxx.spring;

import org.dom4j.Document;
import org.dom4j.DocumentException;
import org.dom4j.Element;
import org.dom4j.XPath;
import org.dom4j.io.SAXReader;
import java.net.URL;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 模拟Spring的实现
 * 1、通过构造器得到相关配置文件
 * 2、通过dom4j解析xml文件，得到List 存放id和class
 * 3、通过反射实例化得到对象 Class.forName(类的全路径).newInstance(); 通过
Map<id,class>存储
 * 4、得到指定的实例化对象
 */
public class MyClassPathXmlApplicationContext implements MyFactory {

    private Map beans = new HashMap(); // 实例化后的对象放入map
    private List<MyBean> myBeans; // 存放已读取bean 配置信息

    /* 1、通过构造器得到相关配置文件 */
    public MyClassPathXmlApplicationContext(String fileName) {

        /* 2、通过dom4j解析xml文件，得到List （存放id和class） */
        this.parseXml(fileName);

        /* 3、通过反射实例化得到对象Class.forName(类路径).newInstance(); 通过Map存储 */
        this.instanceBean();
    }

    /**
     * 通过dom4j解析xml文件，得到List 存放id和class
     * 1、获取解析器
     * 2、得到配置文件的URL

```

```

* 3、通过解析器解析xml文件 (spring.xml)
* 4、通过xpath语法, 获取beans标签下的所有bean标签
* 5、通过指定语法解析文档对象, 返回集合
* 6、判断集合是否为空, 遍历集合
* 7、获取标签元素中的属性
* 8、得到Bean对象, 将Bean对象设置到集合中
* @param fileName
*/
private void parseXml(String fileName) {
    // 1、获取解析器
    SAXReader reader = new SAXReader();
    // 2、得到配置文件的URL
    URL url = this.getClass().getClassLoader().getResource(fileName);
    try {
        // 3、通过解析器解析xml文件 (spring.xml)
        Document document = reader.read(url);
        // 4、通过xpath语法, 获取beans标签下的所有bean标签
        XPath xPath = document.createXPath("beans/bean");
        // 通过指定语法解析文档对象, 返回集合
        List<Element> list = xPath.selectNodes(document);
        // 判断集合是否为空, 遍历集合
        if (list != null && list.size() > 0) {
            myBeans = new ArrayList<>();
            for(Element el : list) {
                // 获取标签元素中的属性
                String id = el.attributeValue("id"); // id 属性值
                String clazz = el.attributeValue("class"); // class 属性值
                System.out.println(el.attributeValue("id"));
                System.out.println(el.attributeValue("class"));
                // 得到Bean对象
                MyBean bean = new MyBean(id, clazz);
                // 将Bean对象设置到集合中
                myBeans.add(bean);
            }
        }
        catch (DocumentException e) {
            e.printStackTrace();
        }
    }
}

/**
 * 通过反射实例化得到对象
 * Class.forName(类的全路径).newInstance();
 * 通过Map<id,class>存储
 */
private void instanceBean() {
    // 判断bean集合是否为空, 不为空遍历得到对应Bean对象
    if (myBeans != null && myBeans.size() > 0) {
        for (MyBean bean : myBeans){
            try {
                // 通过类的全路径实例化对象
                Object object =
Class.forName(bean.getClass()).newInstance();
                // 将id与实例化对象设置到map对象中
                beans.put(bean.getId(), object);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

    }
}

/**
 * 通过key获取map中的指定value
 * @param id
 * @return
 */
@Override
public Object getBean(String id) {
    Object object = beans.get(id);
    return object;
}
}

```

测试自定义 IOC 容器

1. 创建与配置文件中对应的Bean对象

UserService.java

```

package com.xxxx.service;

public class UserService {

    public void test(){
        System.out.println("UserService Test...");
    }
}

```

AccountService.java

```

package com.xxxx.service;

public class AccountService {

    public void test(){
        System.out.println("AccountService Test...");
    }
}

```

2. 测试是否可以获取实例化的Bean对象

```

package com.xxxx;

import com.xxxx.spring.MyFactory;
import com.xxxx.spring.MyClassPathXmlApplicationContext;
import com.xxxx.service.AccountService;
import com.xxxx.service.UserService;

public class App {

    public static void main(String[] args) {
        MyFactory factory = new
        MyClassPathXmlApplicationContext("spring.xml");
    }
}

```

```

// 得到实例化对象
UserService userService = (UserService)
factory.getBean("userService");
userService.test();

UserService userService2 = (UserService)
factory.getBean("userService");
System.out.println(userService+"=====" + userService2);

AccountService accountService =
(AccountService)factory.getBean("accountService");
accountService.test();

}
}

```

Spring 容器在启动的时候 读取xml配置信息，并对配置的 bean 进行实例化（这里模拟的比较简单，仅用于帮助大家理解），同时通过上下文对象提供的 getBean() 方法拿到我们配置的 bean 对象，从而实现外部容器自动化维护并创建 bean 的效果。

Spring IOC 配置文件加载

Spring 配置文件加载

spring.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userService" class="com.xxxx.service.UserService"></bean>
</beans>

```

根据相对路径加载资源

```

ApplicationContext ac = new ClassPathXmlApplicationContext("spring.xml");

```

根据绝对路径加载资源（了解）

```

ApplicationContext ac = new
FileSystemXmlApplicationContext("C:/Ideaworkspace/spring01/src/main/resources/sp
ring.xml");

```

Spring 多配置文件加载

Spring 框架启动时可以加载多个配置文件到环境中。对于比较复杂的项目，可能对应的配置文件有多个，项目在启动部署时会把多个配置文件同时加载进来。

service.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userService" class="com.xxxx.service.UserService"></bean>
</beans>
```

dao.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userDao" class="com.xxxx.dao.UserDao"></bean>
</beans>
```

可变参数，传入多个文件名

```
// 同时加载多个资源文件
ApplicationContext ac = new
ClassPathXmlApplicationContext("spring.xml", "dao.xml");
```

通过总的配置文件import其他配置文件

spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!--导入需要包含的资源文件-->
    <import resource="service.xml"/>
    <import resource="dao.xml"/>
</beans>
```

加载时只需加载总的配置文件即可

```
// 加载总的资源文件
ApplicationContext ac = new ClassPathXmlApplicationContext("spring.xml");
```

Spring IOC 容器 Bean 对象实例化

构造器实例化

注：通过默认构造器创建 空构造方法必须存在 否则创建失败

1. 设置配置文件 spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userService" class="com.xxxx.service.UserService"></bean>

</beans>
```

2. 获取实例化对象

```
ApplicationContext ac = new ClassPathXmlApplicationContext("spring.xml");
UserService userService = (UserService) ac.getBean("userService");
userService.test();
```

静态工厂实例化（了解）

注：

- 要有该工厂类及工厂方法
- 工厂方法为静态的

1. 定义静态工厂类

```
package com.xxxx.factory;

import com.xxxx.service.UserService;

/**
 * 定义静态工厂类
 */
public class StaticFactory {
    /**
     * 定义对应的静态方法，返回实例化对象
     * @return
     */
    public static UserService createUserService() {
        return new UserService();
    }
}
```

2. 设置配置文件 spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!--静态工厂-->
    <bean id="userService" class="com.xxxx.factory.StaticFactory" factory-
method="createUserService"></bean>

</beans>
```

3. 获取实例化对象

```
ApplicationContext ac = new ClassPathXmlApplicationContext("spring.xml");
UserService userService = (UserService) ac.getBean("userService");
userService.test();
```

当我们指定Spring使用静态工厂方法来创建Bean实例时，Spring将先解析配置文件，并根据配置文件指定的信息，**通过反射调用静态工厂类的静态工厂方法，并将该静态工厂方法的返回值作为Bean实例**，在这个过程中，Spring不再负责创建Bean实例，**Bean实例是由用户提供的静态工厂方法提供的**。

实例化工厂实例化（了解）

注：

- 工厂方法为非静态方法
- 需要配置工厂bean，并在业务bean中配置factory-bean，factory-method属性

1. 定义工厂类

```
package com.xxxx.factory;

import com.xxxx.service.UserService;

/**
 * 定义工厂类
 */
public class InstanceFactory {
    /**
     * 定义方法，返回实例化对象
     * @return
     */
    public UserService createUserService() {
        return new UserService();
    }
}
```

2. 设置配置文件 spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!--
        实例化工厂
        1. 定义实例化工厂bean
        2. 引用工厂bean 指定工厂创建方法(方法为非静态)
    -->
    <bean id="instanceFactory" class="com.xxxx.factory.InstanceFactory">
</bean>
    <bean id="userService" factory-bean="instanceFactory" factory-
method="createUserService"></bean>

</beans>

```

3. 获取实例化对象

```

ApplicationContext ac = new ClassPathXmlApplicationContext("spring.xml");
UserService userService = (UserService) ac.getBean("userService");
userService.test();

```

Spring三种实例化Bean的方式比较

- 方式一：通过bean的缺省构造函数创建，当各个bean的业务逻辑相互比较独立的时候或者和外界关联较少的时候可以使用。
- 方式二：利用静态factory方法创建，可以统一管理各个bean的创建，如各个bean在创建之前需要相同的初始化处理，则可用这个factory方法进行统一的处理等等。
- 方式三：利用实例化factory方法创建，即将factory方法也作为业务bean来控制，1可用于集成其他框架的bean创建管理方法，2能够使bean和factory的角色互换。

开发中项目一般使用一种方式实例化bean，项目开发基本采用第一种方式，交给Spring托管，使用时直接拿来使用即可。另外两种了解

Spring IOC 注入

手动实例化与外部引入

图一：

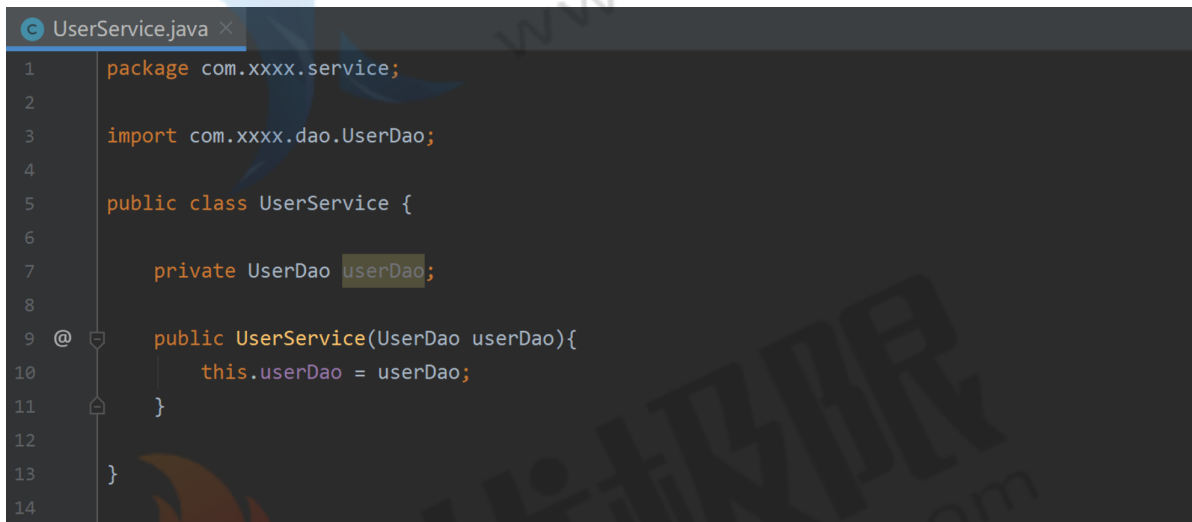


```

1 package com.xxxx.service;
2
3 import com.xxxx.dao.UserDao;
4
5 public class UserService {
6
7     UserDao userDao = new UserDao();
8
9
10 }
11

```

图二:



```

1 package com.xxxx.service;
2
3 import com.xxxx.dao.UserDao;
4
5 public class UserService {
6
7     private UserDao userDao;
8
9     @
10     public UserService(UserDao userDao){
11         this.userDao = userDao;
12     }
13
14 }
15

```

对比发现：图二中对于 UserDao 对象的创建并没有像图一那样主动的去实例化，而是通过带参方法形式将 UserDao 传入过来，从而实现 UserService 对 UserDao 类的依赖。

而实际创建对象的幕后对象即是交给了外部来创建。

Spring IOC 手动装配（注入）

Spring 支持的注入方式共有四种：set 注入、构造器注入、静态工厂注入、实例化工厂注入。

set方法注入

注：

- 属性字段需要提供set方法
- 四种方式，推荐使用set方法注入

业务对象 JavaBean

1. 属性字段提供set方法

```
public class UserService {

    // 业务对象UserDao set注入（提供set方法）
    private UserDao userDao;
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }
}
```

2. 配置文件的bean标签设置property标签

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!--
        IOC通过property标签手动装配（注入）：
        Set方法注入
        name: bean对象中属性字段的名称
        ref: 指定bean标签的id属性值
    -->
    <bean id="userDao" class="com.xxxx.dao.UserDao"></bean>
    <bean id="userService" class="com.xxxx.service.UserService">
        <!--业务对象 注入-->
        <property name="userDao" ref="userDao"/>
    </bean>
</beans>
```

常用对象和基本类型

1. 属性字段提供set方法

```
public class UserService {

    // 常用对象String set注入（提供set方法）
    private String host;
    public void setHost(String host) {
        this.host = host;
    }

    // 基本类型Integer set注入（提供set方法）
    private Integer port;
    public void setPort(Integer port) {
        this.port = port;
    }
}
```

2. 配置文件的bean标签设置property标签

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
https://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<!--
```

IOC通过property标签手动装配（注入）：

Set方法注入

name: bean对象中属性字段的名称

value: 具体的值（基本类型 常用对象|日期 集合）

```
-->
```

```
<bean id="userService" class="com.xxxx.service.UserService">
```

```
<!--常用对象String 注入-->
```

```
<property name="host" value="127.0.0.1"/>
```

```
<!--基本类型注入-->
```

```
<property name="port" value="8080"/>
```

```
</bean>
```

```
</beans>
```

集合类型和属性对象

1. 属性字段提供set方法

```
public class UserService {  
  
    // List集合 set注入（提供set方法）  
    public List<String> list;  
    public void setList(List<String> list) {  
        this.list = list;  
    }  
  
    // Set集合 set注入（提供set方法）  
    private Set<String> set;  
    public void setSet(Set<String> set) {  
        this.set = set;  
    }  
  
    // Map set注入（提供set方法）  
    private Map<String, Object> map;  
    public void setMap(Map<String, Object> map) {  
        this.map = map;  
    }  
  
    // Properties set注入（提供set方法）  
    private Properties properties;  
    public void setProperties(Properties properties) {  
        this.properties = properties;  
    }  
}
```

2. 配置文件的bean标签设置property标签

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<!--
```

IOC通过property标签手动装配（注入）：

Set方法注入

name: bean对象中属性字段的名称

value: 具体的值（基本类型 常用对象|日期 集合）

```
-->
```

```
<!--List集合 注入-->
```

```
<property name="list">
```

```
<list>
```

```
<value>上海</value>
```

```
<value>北京</value>
```

```
<value>杭州</value>
```

```
</list>
```

```
</property>
```

```
<!--Set集合注入-->
```

```
<property name="set">
```

```
<set>
```

```
<value>上海SH</value>
```

```
<value>北京BJ</value>
```

```
<value>杭州HZ</value>
```

```
</set>
```

```
</property>
```

```
<!--Map注入-->
```

```
<property name="map">
```

```
<map>
```

```
<entry>
```

```
<key><value>周杰伦</value></key>
```

```
<value>我是如此相信</value>
```

```
</entry>
```

```
<entry>
```

```
<key><value>林俊杰</value></key>
```

```
<value>可惜没如果</value>
```

```
</entry>
```

```
<entry>
```

```
<key><value>陈奕迅</value></key>
```

```
<value>十年</value>
```

```
</entry>
```

```
</map>
```

```
</property>
```

```
<!--Properties注入-->
```

```
<property name="properties">
```

```
<props>
```

```
<prop key="上海">东方明珠</prop>
```

```
<prop key="北京">天安门</prop>
```

```
<prop key="杭州">西湖</prop>
```

```
</props>
```

```
</property>
```

```
</beans>
```

测试代码

UserService.java

```
public class UserService {

    // 业务对象UserDao set注入（提供set方法）
    private UserDao userDao;
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    // 常用对象String set注入（提供set方法）
    private String host;
    public void setHost(String host) {
        this.host = host;
    }

    // 基本类型Integer set注入（提供set方法）
    private Integer port;
    public void setPort(Integer port) {
        this.port = port;
    }

    // List集合 set注入（提供set方法）
    public List<String> list;
    public void setList(List<String> list) {
        this.list = list;
    }
    // List集合输出
    public void printList() {
        list.forEach(s -> System.out.println(s));
    }

    // Set集合 set注入（提供set方法）
    private Set<String> set;
    public void setSet(Set<String> set) {
        this.set = set;
    }
    // Set集合输出
    public void printSet() {
        set.forEach(s -> System.out.println(s));
    }

    // Map set注入（提供set方法）
    private Map<String, Object> map;
    public void setMap(Map<String, Object> map) {
        this.map = map;
    }
    // Map输出
    public void printMap() {
        map.forEach((k,v) -> System.out.println(k + ", " + v));
    }

    // Properties set注入（提供set方法）
}
```

```

private Properties properties;
public void setProperties(Properties properties) {
    this.properties = properties;
}
// Properties输出
public void printProperties(){
    properties.forEach((k,v) -> System.out.println(k + ", " + v ));
}

public void test(){
    System.out.println("UserService Test...");

    userDao.test();

    studentDao.test();

    System.out.println("Host: " + host + ", port: " + port);

    // List集合
    printList();

    // Set集合
    printSet();

    // Map
    printMap();

    // Properties
    printProperties();
}
}

```

spring.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!--
        IOC通过property标签手动装配（注入）：
        Set方法注入
        name: bean对象中属性字段的名称
        ref: 指定bean标签的id属性值
        value: 具体的值（基本类型 常用对象|日期 集合）
    -->

    <bean id="userDao" class="com.xxxx.dao.UserDao"></bean>
    <bean id="userService" class="com.xxxx.service.UserService">
        <!--业务对象 注入-->
        <property name="userDao" ref="userDao"/>
        <property name="studentDao" ref="studentDao"/>
    
```

```
<!--常用对象String 注入-->
<property name="host" value="192.168.1.109"/>
<!--基本类型注入-->
<property name="port" value="8080"/>

<!--List集合 注入-->
<property name="list">
    <list>
        <value>上海</value>
        <value>北京</value>
        <value>杭州</value>
    </list>
</property>

<!--Set集合注入-->
<property name="set">
    <set>
        <value>上海SH</value>
        <value>北京BJ</value>
        <value>杭州HZ</value>
    </set>
</property>

<!--Map注入-->
<property name="map">
    <map>
        <entry>
            <key><value>周杰伦</value></key>
            <value>我是如此相信</value>
        </entry>
        <entry>
            <key><value>林俊杰</value></key>
            <value>可惜没如果</value>
        </entry>
        <entry>
            <key><value>陈奕迅</value></key>
            <value>十年</value>
        </entry>
    </map>
</property>

<!--Properties注入-->
<property name="properties">
    <props>
        <prop key="上海">东方明珠</prop>
        <prop key="北京">天安门</prop>
        <prop key="杭州">西湖</prop>
    </props>
</property>

</bean>

</beans>
```

构造器注入

注:

- 提供带参构造器

单个Bean对象作为参数

Java 代码

```
public class UserService {  
  
    private UserDao userDao; // JavaBean 对象  
  
    public UserService(UserDao userDao) {  
        this.userDao = userDao;  
    }  
  
    public void test(){  
        System.out.println("UserService Test...");  
  
        userDao.test();  
    }  
  
}
```

XML配置

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://www.springframework.org/schema/beans  
        https://www.springframework.org/schema/beans/spring-beans.xsd">  
    <!--  
        IOC通过构造器注入:  
        通过constructor-arg标签进行注入  
        name: 属性名称  
        ref: 指定bean标签的id属性值  
    -->  
    <bean id="userDao" class="com.xxxx.dao.UserDao" ></bean>  
  
    <bean id="userService" class="com.xxxx.service.UserService">  
        <constructor-arg name="userDao" ref="userDao"></constructor-arg>  
    </bean>  
  
</beans>
```

多个Bean对象作为参数

Java 代码

```
public class UserService {  
  
    private UserDao userDao; // JavaBean 对象  
    private AccountDao accountDao // JavaBean 对象
```

```

public UserService(UserDao userDao, AccountDao accountDao) {
    this.userDao = userDao;
    this.accountDao = accountDao;
}

public void test(){
    System.out.println("UserService Test...");

    userDao.test();
    accountDao.test();
}
}

```

XML配置

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">
    <!--
        IOC通过构造器注入：
            通过constructor-arg标签进行注入
            name: 属性名称
            ref: 指定bean标签的id属性值
    -->
    <bean id="userDao" class="com.xxxx.dao.UserDao" ></bean>
    <bean id="accountDao" class="com.xxxx.dao.AccountDao" ></bean>

    <bean id="userService" class="com.xxxx.service.UserService">
        <constructor-arg name="userDao" ref="userDao"></constructor-arg>
        <constructor-arg name="accountDao" ref="accountDao"></constructor-arg>
    </bean>

</beans>

```

Bean对象和常用对象作为参数

Java 代码

```

public class UserService {

    private UserDao userDao; // JavaBean 对象
    private AccountDao accountDao; // JavaBean 对象
    private String uname; // 字符串类型

    public UserService(UserDao userDao, AccountDao accountDao, String uname) {
        this.userDao = userDao;
        this.accountDao = accountDao;
        this.uname = uname;
    }

    public void test(){

```

```

        System.out.println("UserService Test...");

        userDao.test();
        accountDao.test();
        System.out.println("uname: " + uname);
    }
}

```

XML配置

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">
    <!--
        IOC通过构造器注入：
            通过constructor-arg标签进行注入
            name: 属性名称
            ref: 指定bean标签的id属性值
            value: 基本类型 常用对象的值
            index: 构造器中参数的下标，从0开始
    -->
    <bean id="userDao" class="com.xxxx.dao.UserDao" ></bean>
    <bean id="accountDao" class="com.xxxx.dao.AccountDao" ></bean>
    <bean id="userService" class="com.xxxx.service.UserService">
        <constructor-arg name="userDao" ref="userDao"></constructor-arg>
        <constructor-arg name="accountDao" ref="accountDao"></constructor-arg>
        <constructor-arg name="uname" value="admin"></constructor-arg>
    </bean>
</beans>

```

循环依赖问题

循环问题产生的原因：

Bean通过构造器注入，之间彼此相互依赖对方导致bean无法实例化。

问题展示：

1. Java 代码

```

public class AccountService {

    private RoleService roleService;

    public AccountService(RoleService roleService) {
        this.roleService = roleService;
    }

    public void test() {
        System.out.println("AccountService Test...");
    }
}

```

```

public class RoleService {

    private AccountService accountService;

    public RoleService(AccountService accountService) {
        this.accountService = accountService;
    }

    public void test() {
        System.out.println("RoleService Test...");
    }

}

```

2. XML配置

```

<!--
    如果多个bean对象中互相注入，则会出现循环依赖的问题
    可以通过set方法注入解决
-->
<bean id="accountService" class="com.xxxx.service.AccountService">
    <constructor-arg name="roleService" ref="roleService"/>
</bean>

<bean id="roleService" class="com.xxxx.service.RoleService">
    <constructor-arg name="accountService" ref="accountService"/>
</bean>

```

如何解决：将构造器注入改为set方法注入

1. Java代码

```

public class AccountService {

    private RoleService roleService;

    /* public AccountService(RoleService roleService) {
        this.roleService = roleService;
    } */

    public void setRoleService(RoleService roleService) {
        this.roleService = roleService;
    }

    public void test() {
        System.out.println("AccountService Test...");
    }

}

public class RoleService {

    private AccountService accountService;

    /* public RoleService(AccountService accountService) {
        this.accountService = accountService;
    } */

```

```

    }*/

    public void setAccountService(AccountService accountService) {
        this.accountService = accountService;
    }

    public void test() {
        System.out.println("RoleService Test...");
    }
}

```

2. XML配置

```

<!--
    <bean id="accountService" class="com.xxxx.service.AccountService">
        <constructor-arg name="roleService" ref="roleService"/>
    </bean>

    <bean id="roleService" class="com.xxxx.service.RoleService">
        <constructor-arg name="accountService" ref="accountService"/>
    </bean>
-->
<!--修改为set方法注入-->
<bean id="accountService" class="com.xxxx.service.AccountService">
    <property name="roleService" ref="roleService"/>
</bean>

<bean id="roleService" class="com.xxxx.service.RoleService">
    <property name="accountService" ref="accountService"/>
</bean>

```

静态工厂注入

1. 定义静态工厂类

```

public class StaticFactory {

    // 定义静态方法
    public static TypeDao createTypeDao() {
        return new TypeDao();
    }
}

```

2. Java代码

```

public class TypeService {

    private TypeDao typeDao;

    public void setTypeDao(TypeDao typeDao) {
        this.typeDao = typeDao;
    }

    public void test() {
        System.out.println("TypeService Test...");
    }

}

```

3. XML配置

在配置文件中设置bean标签，指定工厂对象并设置对应的方法

```

<bean id="typeService" class="com.xxxx.service.TypeService">
    <property name="typeDao" ref="typeDao"/>
</bean>
<!--

```

静态工厂注入：

静态工厂注入也是借助set方法注入，只是被注入的bean对象的实例化是通过静态工厂实例化的

```

-->
<bean id="typeDao" class="com.xxxx.factory.StaticFactory" factory-
method="createTypeDao"></bean>

```

实例化工厂注入

1. 定义工厂类

```

public class InstanceFactory {
    public TypeDao createTypeDao() {
        return new TypeDao();
    }
}

```

2. Java代码

```

public class TypeService {

    private TypeDao typeDao;

    public void setTypeDao(TypeDao typeDao) {
        this.typeDao = typeDao;
    }

    public void test() {
        System.out.println("TypeService Test...");
    }

}

```

3. XML配置

声明工厂bean标签，声明bean对象，指明工厂对象和工厂方法

```
<bean id="typeService" class="com.xxxx.service.TypeService">
    <property name="typeDao" ref="typeDao"/>
</bean>
<!--
    实例化工厂注入：
    实例化工厂注入也是借助set方法注入，只是被注入的bean对象的实例化是通过实例化工厂实
    例化的
-->
<bean id="instanceFactory" class="com.xxxx.factory.InstanceFactory"></bean>
<bean id="typeDao" factory-bean="instanceFactory" factory-
method="createTypeDao"></bean>
```

重点掌握set注入和构造器注入，工厂方式了解即可。实际开发中基本使用set方式注入bean。

注入方式的选择

开发项目中set方式注入首选

使用构造注入可以在构建对象的同时一并完成依赖关系的建立，对象一建立则所有的一切也就准备好了，但如果要建立的对象关系很多，使用构造器注入会在构造函数上留下一长串的参数，且不易记忆，这时使用Set注入会是个不错的选择。

使用Set注入可以有明确的名称，可以了解注入的对象会是什么，像setXXX()这样的名称会比记忆Constructor上某个参数的位置代表某个对象更好。

p名称空间的使用

spring2.5以后，为了简化setter方法属性注入，引用p名称空间的概念，可以将子元素，简化为元素属性配置。

1. 属性字段提供 set 方法

```
public class UserService {

    // 业务对象UserDao set注入（提供set方法）
    private UserDao userDao;
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    // 常用对象String set注入（提供set方法）
    private String host;
    public void setHost(String host) {
        this.host = host;
    }
}
```

2. 在配置文件 spring.xml 引入 p 名称空间

```
xmlns:p="http://www.springframework.org/schema/p"
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:p="http://www.springframework.org/schema/p"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userDao" class="com.xxxx.dao.UserDao"></bean>
    <!--
        p:属性名:="xxx"          引入常量值
        p:属性名-ref:="xxx"       引入其他Bean对象的id属性值
    -->
    <bean id="userService" class="com.xxxx.service.UserService"
          p:userDao-ref="userDao"
          p:host="127.0.0.1" />

</beans>

```

Spring IOC 自动装配（注入）

注解方式注入 Bean

对于 bean 的注入，除了使用 xml 配置以外，可以使用注解配置。注解的配置，可以简化配置文件，提高开发的速度，使程序看上去更简洁。对于注解的解释，Spring 对于注解有专门的解释器，对定义的注解进行解析，实现对应 bean 对象的注入。通过**反射技术实现**。

准备环境

1. 修改配置文件

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

```

2. 开启自动化注入

```

<!--开启自动化装配（注入）-->
<context:annotation-config/>

<bean id="userDao" class="com.xxxx.dao.UserDao"></bean>
<bean id="userService" class="com.xxxx.service.UserService"></bean>

```

3. 给注入的bean对象添加注解

@Resource注解

- 名称查找对应的bean对象（属性字段的名称与bean标签的id属性值相等）

```
1. <!-- https://mvnrepository.com/artifact/javax.annotation/javax.annotation-api -->
    <dependency>
        <groupId>javax.annotation</groupId>
        <artifactId>javax.annotation-api</artifactId>
        <version>1.3.2</version>
    </dependency>
/**
 * @Resource注解实现自动注入（反射）
 * 默认根据属性字段名称查找对应的bean对象（属性字段的名称与bean标签的id属性值相等）
 */
public class UserService {

    @Resource
    private UserDao userDao; // 属性字段的名称与bean标签的id属性值相等

    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void test() {
        // 调用UserDao的方法
        userDao.test();
    }
}
```

2. 如果属性字段名称未找到，则会通过类型（Class类型）查找

```
/**
 * @Resource注解实现自动注入（反射）
 * 如果属性字段名称未找到，则会通过类型（Class类型）查找
 */
public class UserService {

    @Resource
    private UserDao ud; // 当在配置文件中属性字段名（ud）未找到，则会查找对应的class（UserDao类型）

    public void setUd(UserDao ud) {
        this.ud = ud;
    }

    public void test() {
        // 调用UserDao的方法
        ud.test();
    }
}
```

3. 属性可以提供set方法，也可以不提供set方法

```
/**
 * @Resource注解实现自动注入（反射）
```

```

    * 属性可以提供set方法，也可以不提供set方法
    */
    public class UserService {

        @Resource
        private UserDao userDao; // 不提供set方法

        public void test() {
            // 调用UserDao的方法
            userDao.test();
        }
    }
}

```

4. 注解可以声明在属性级别 或 set方法级别

```

/**
 * @Resource注解实现自动注入（反射）
 * 注解可以声明在属性级别 或 set方法级别
 */
    public class UserService {

        private UserDao userDao;

        @Resource // 注解也可设置在set方法上
        public void setUserDao(UserDao userDao) {
            this.userDao = userDao;
        }

        public void test() {
            // 调用UserDao的方法
            userDao.test();
        }
    }
}

```

5. 可以设置name属性，name属性值必须与bean标签的id属性值一致；如果设置了name属性值，就只会按照name属性值查找bean对象

```

/**
 * @Resource注解实现自动注入（反射）
 * 可以设置name属性，name属性值必须与bean的id属性值一致；
 * 如果设置了name属性值，就只会按照name属性值查找bean对象
 */
    public class UserService {

        @Resource(name = "userDao") // name属性值与配置文件中bean标签的id属性值一致
        private UserDao ud;

        public void test() {
            // 调用UserDao的方法
            ud.test();
        }
    }
}

```

6. 当注入接口时，如果接口只有一个实现则正常实例化；如果接口存在多个实现，则需要使用name属性指定需要被实例化的bean对象

定义接口类 IUserDao.java

```
package com.xxxx.dao;

/**
 * 定义接口类
 */
public interface IUserDao {
    public void test();
}
```

定义接口实现类 UserDao01.java

```
package com.xxxx.dao;

/**
 * 接口实现类
 */
public class UserDao01 implements IUserDao {

    @Override
    public void test(){
        System.out.println("UserDao01...");
    }
}
```

定义接口实现类 UserDao02.java

```
package com.xxxx.dao;

/**
 * 接口实现类
 */
public class UserDao02 implements IUserDao {

    @Override
    public void test(){
        System.out.println("UserDao02...");
    }
}
```

XML配置文件

```
<!--开启自动化装配（注入）-->
<context:annotation-config/>

<bean id="userService" class="com.xxxx.service.UserService"></bean>

<bean id="userDao01" class="com.xxxx.dao.UserDao01"></bean>
<bean id="userDao02" class="com.xxxx.dao.UserDao02"></bean>
```

使用注解 UserService.java

```

/**
 * @Resource注解实现自动注入（反射）
 * 当注入接口时，如果接口只有一个实现则正常实例化；如果接口存在多个实现，则需要使用name
属性指定需要被实例化的bean对象
 */
public class UserService {

    @Resource(name = "userDao01") // name属性值与其中一个实现类的bean标签的id属性
    值一致
    private IUserDao iUserDao; // 注入接口（接口存在多个实现）

    public void test() {
        iUserDao.test();
    }
}

```

@Autowired注解

@Autowired注解实现自动化注入：

- 默认通过类型（Class类型）查找bean对象 与属性字段的名称无关
- 属性可以提供set方法，也可以不提供set方法
- 注解可以声明在属性级别 或 set方法级别
- 可以添加@Qualifier结合使用，通过value属性值查找bean对象（value属性值必须要设置，且值要与bean标签的id属性值对应）

1. 默认通过类型（Class类型）查找bean对象 与属性字段的名称无关

```

/**
 * @Autowired注解实现自动化注入
 * 默认通过类型（Class类型）查找bean对象 与属性字段的名称无关
 */
public class UserService {

    @Autowired
    private UserDao userDao; // 默认通过类型（Class类型）查找bean对象 与属性字段的
    名称无关

    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void test() {
        // 调用UserDao的方法
        userDao.test();
    }
}

```

2. 属性可以提供set方法，也可以不提供set方法

```

/**
 * @Autowired注解实现自动化注入
 * 属性可以提供set方法，也可以不提供set方法
 */

```

```

public class UserService {

    @Autowired
    private UserDao userDao; // 不提供set方法

    public void test() {
        // 调用UserDao的方法
        userDao.test();
    }

}

```

3. 注解可以声明在属性级别 或 set方法级别

```

/**
 * @Autowired注解实现自动化注入
 * 注解可以声明在属性级别 或 set方法级别
 */
public class UserService {

    private UserDao userDao;

    @Autowired// 注解可以声明在set方法级别
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void test() {
        // 调用UserDao的方法
        userDao.test();
    }

}

```

4. 可以添加@Qualifier结合使用，通过value属性值查找bean对象（value属性值必须要设置，且值要与bean标签的id属性值对应）

```

/**
 * @Autowired注解实现自动化注入
 * 可以添加@Qualifier结合使用，通过value属性值查找bean对象
 * value属性值必须要设置，且值要与bean标签的id属性值对应
 */
public class UserService {

    @Autowired
    @Qualifier(value="userDao") // value属性值必须要设置，且值要与bean标签的id属性值对应
    private UserDao userDao;

    public void test() {
        userDao.test();
    }

}

```

推荐使用@Resource 注解是属于J2EE的，减少了与Spring的耦合。

Spring IOC 扫描器

实际的开发中，bean的数量非常多，采用手动配置bean的方式已无法满足生产需要，Spring这时候同样提供了扫描的方式，对扫描到的bean对象统一进行管理，简化开发配置，提高开发效率。

Spring IOC 扫描器的配置

Spring IOC 扫描器

作用：bean对象统一进行管理，简化开发配置，提高开发效率

1、设置自动化扫描的范围

如果bean对象未在指定包范围，即使声明了注解，也无法实例化

2、使用指定的注解（声明在类级别） bean对象的id属性默认是 类的首字母小写

Dao层：

@Repository

Service层：

@Service

Controller层：

@Controller

任意类：

@Component

注：开发过程中建议按照指定规则声明注解

1. 设置自动化扫描范围

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <!-- 设置自动化扫描的范围 -->
    <context:component-scan base-package="com.xxxx"/>

</beans>
```

2. 使用特定的注解

@Repository （Dao层）

```
@Repository
public class ResourceDao {

    public void test() {
        System.out.println("ResourceDao...");
    }
}
```

@Service (Service层)

```

@Service
public class ResourceService {

    @Resource
    private ResourceDao resourceDao; // service层注入dao层的bean对象

    public void test() {
        System.out.println("ResourceService...");
        resourceDao.test();
    }
}

```

@Controller (Controller 层)

```

@Controller
public class ResourceController {

    @Autowired
    private ResourceService resourceService; // Controller层注入service层的bean对象

    public void test() {
        System.out.println("ResourceController...");
        resourceService.test();
    }
}

```

@Component (任意层)

```

@Component
public class PropertyUtils {
    public void test(){
        System.out.println("PropertyUtils...");
    }
}

```

Spring 模拟用户登录流程

Dao层 (查询用户记录)

1. 定义JavaBean User.java

```

package com.xxxx.po;

/**
 * User 用户实体类
 */
public class User {

    private String userName; // 用户名称
    private String userPwd; // 用户密码

    public String getUserName() {

```

```

        return userName;
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }

    public String getUserPwd() {
        return userPwd;
    }

    public void setUserPwd(String userPwd) {
        this.userPwd = userPwd;
    }
}

```

2. 编写Dao层 UserDao.java

```

package com.xxxx.dao;

import com.xxxx.po.User;
import org.springframework.stereotype.Repository;

@Repository
public class UserDao {

    private final String USERNAME = "admin";
    private final String USERPWD = "admin";

    /**
     * 通过用户名称查询用户对象
     * @param userName
     * @return
     */
    public User queryUserByUserName(String userName){
        User user = null;
        // 判断用户名称是否正确
        if(!USERNAME.equals(userName)){
            // 如果不正确，返回null
            return null;
        }
        // 如果正确，将用户名称和密码设置到user对象中
        user = new User();
        user.setUserName(USERNAME);
        user.setUserPwd(USERPWD);

        return user;
    }
}

```

Service层（业务逻辑处理）

1. 定义业务处理返回消息模型 MessageModel.java

```
package com.xxxx.po.vo;

/**
 * 定义业务处理返回消息模型
 * 封装返回结果
 */
public class MessageModel {

    private Integer resultCode = 1; // 结果状态码 1=成功, 0=失败
    private String resultMsg = "操作成功!"; // 结果提示信息

    public Integer getResultCode() {
        return resultCode;
    }

    public void setResultCode(Integer resultCode) {
        this.resultCode = resultCode;
    }

    public String getResultMsg() {
        return resultMsg;
    }

    public void setResultMsg(String resultMsg) {
        this.resultMsg = resultMsg;
    }
}
```

2. 编写Service层 UserService.java

```
package com.xxxx.service;

import com.xxxx.dao.UserDao1;
import com.xxxx.po.User;
import com.xxxx.po.vo.MessageModel;
import org.springframework.stereotype.Service;

import javax.annotation.Resource;

@Service
public class UserService {

    @Resource
    private UserDao userDao;

    /**
     * 验证用户登录
     * @param userName
     * @param userPwd
     * @return
     */
    public MessageModel userLoginCheck(String userName, String userPwd){
        // 定义业务处理返回消息模型
        MessageModel messageModel = new MessageModel();
    }
}
```

```

// 判断用户名称是否非空
if(null == userName || "".equals(userName.trim())){
    messageModel.setResultCode(0);
    messageModel.setResultMsg("用户名不能为空!");
    return messageModel;
}
// 判断用户密码是否为空
if(null == userPwd || "".equals(userPwd.trim())){
    messageModel.setResultCode(0);
    messageModel.setResultMsg("密码不能为空!");
    return messageModel;
}
// 通过用户名称查询用户对象
User user = userDao.queryUserByUserName(userName);
// 判断用户对象是否为空
if(null == user){
    messageModel.setResultCode(0);
    messageModel.setResultMsg("该用户不存在!");
    return messageModel;
}
// 如果用户对象不为空, 判断密码是否正确
if(!user.getUserPwd().equals(userPwd)){
    messageModel.setResultCode(0);
    messageModel.setResultMsg("用户密码不正确!");
    return messageModel;
}
// 登录成功
messageModel.setResultMsg("登录成功!");

return messageModel;
}
}

```

Controller层 (接收请求)

1. 编写Controller层 UserController.java

```

package com.xxxx.controller;

import com.xxxx.po.vo.MessageModel;
import com.xxxx.service.UserService1;
import org.springframework.stereotype.Controller;

import javax.annotation.Resource;

@Controller
public class UserController {
    @Resource
    private UserService userService;

    /**
     * 用户登录
     * @param userName
     * @param userPwd
     * @return
     */
}

```

```

    */
    public MessageModel login(String userName, String userPwd){
        // 调用Dao层判断用户登录操作，返回结果
        MessageModel messageModel = userService.userLoginCheck(userName,
userPwd);
        return messageModel;
    }
}

```

通过 JUnit 进行测试

```

package com.xxxx;

import com.xxxx.controller.UserController;
import com.xxxx.po.vo.MessageModel;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class TestLogin {

    @Test
    public void test() {
        // 得到Spring容器上下文环境
        ApplicationContext ac = new
ClassPathXmlApplicationContext("spring.xml");
        // 得到UserController实例化对象
        UserController userController = (UserController)
ac.getBean("userController");
        // 传入参数调用UserController的方法，返回封装类
        MessageModel messageModel= userController.login("admin", "admin");

        System.out.println("状态码: " + messageModel.getResultCode() + ", 提示信
息: " + messageModel.getResultMsg());
    }
}

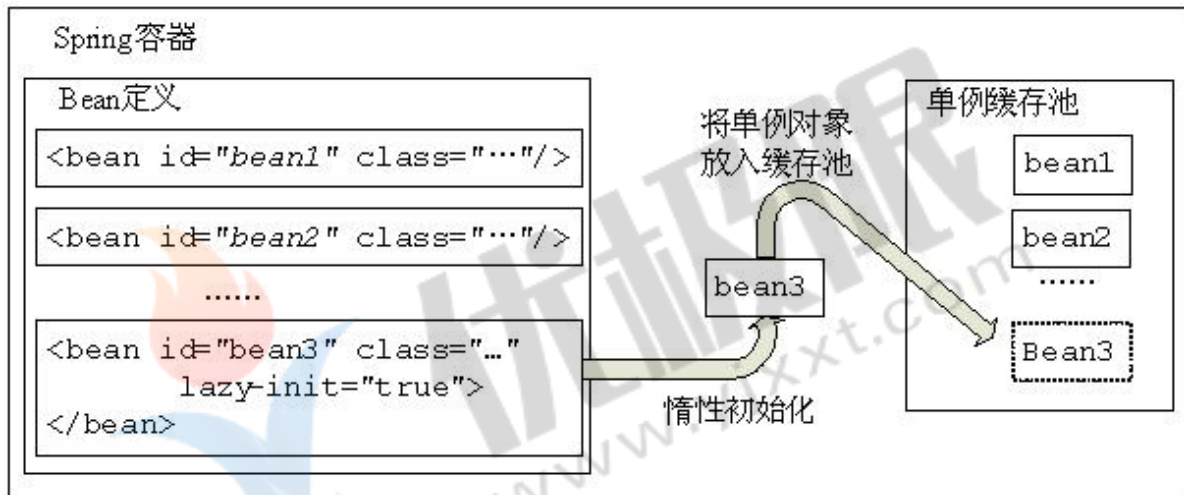
```

Bean的作用域与生命周期

Bean的作用域

默认情况下，我们从Spring容器中拿到的对象均是**单例**的，对于bean的作用域类型如下：

singleton 作用域



注意: lazy-init是懒加载, 如果等于true时作用是指Spring容器启动的时候不会去实例化这个bean, 而是在程序调用时才去实例化. 默认是false即Spring容器启动时实例化.

默认情况下, 被管理的bean只会IOC容器中存在一个实例, 对于所有获取该Bean的操作Spring容器将只返回同一个Bean。

容器在启动的情况下就实例化所有singleton 的 bean对象, 并缓存与容器中

lazy-init属性 (懒加载)

如果为false, 则在IOC容器启动时会实例化bean对象, 默认false

如果为true, 则IOC容器启动时不会实例化Bean对象, 在使用bean对象时才会实例化

lazy-init设置为false有什么好处?

- 1) 可以提前发现潜在的配置问题
- 2) Bean 对象存在于缓存中, 使用时不用再去实例化bean, 加快程序运行效率

什么对象适合作为单例对象?

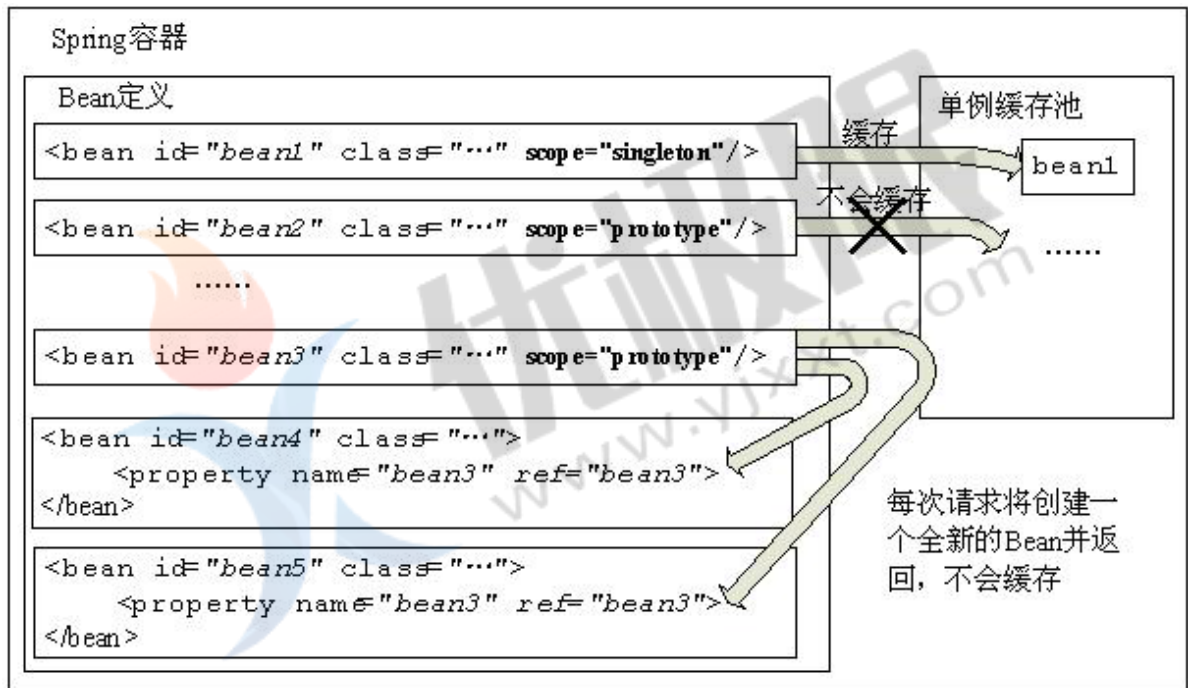
一般来说对于无状态或状态不可改变的对象适合使用单例模式。(不存在会改变对象状态的成员变量)

比如: controller层、service层、dao层

什么是无状态或状态不可改变的对象?

实际上对象状态的变化往往均是由于属性值得变化而引起的, 比如user类 姓名属性会有变化, 属性姓名的变化一般会引起user对象状态的变化。对于我们的程序来说, 无状态对象没有实例变量的存在, 保证了线程的安全性, service 层业务对象即是无状态对象。线程安全的。

prototype 作用域



通过scope="prototype" 设置bean的类型，每次向Spring容器请求获取Bean都返回一个全新的Bean，相对于"singleton"来说就是不缓存Bean，每次都是一个根据Bean定义创建的全新Bean。

Web应用中的作用域

1. request作用域

表示每个请求需要容器创建一个全新Bean。比如提交表单的数据必须是对每次请求新建一个Bean来保持这些表单数据，请求结束释放这些数据。

2. session作用域

表示每个会话需要容器创建一个全新Bean。比如对于每个用户一般会有一个会话，该用户的用户信息需要存储到会话中，此时可以将该Bean作用域配置为session级别。

3. globalSession作用域

类似于session作用域，其用于portlet(Portlet是基于Java的Web组件，由Portlet容器管理，并由容器处理请求，生产动态内容)环境的web应用。如果在非portlet环境将视为session作用域。

配置方式和基本的作用域相同，只是必须要有web环境支持，并配置相应的容器监听器或拦截器从而能应用这些作用域，目前先熟悉概念，后续集成web时讲解具体使用，大家只需要知道有这些作用域就可以了。

Bean的生命周期

对比已经学过的servlet 生命周期（容器启动装载并实例化servlet类，初始化servlet，调用service方法，销毁servlet）。

同样对于Spring容器管理的bean也存在生命周期的概念

在Spring中，Bean的生命周期包括Bean的定义、初始化、使用和销毁4个阶段

Bean的定义

在Spring中，通常是通过配置文档的方式来定义Bean的。

在一个配置文档中，可以定义多个Bean。

Bean 的初始化

默认在IOC容器加载时，实例化对象。

Spring bean 初始化有两种方式：

方式一：在配置文档中通过指定 init-method 属性来完成。

```
public class RoleService {  
    // 定义初始化时需要被调用的方法  
    public void init() {  
        System.out.println("RoleService init...");  
    }  
}
```

```
<!-- 通过init-method属性指定方法 -->  
<bean id="roleService" class="com.xxxx.service.RoleService" init-method="init">  
</bean>
```

方式二：实现 org.springframework.beans.factory.InitializingBean 接口。

```
public class RoleService implements InitializingBean {  
  
    @Override  
    public void afterPropertiesSet() throws Exception {  
        System.out.println("RoleService init...");  
    }  
}
```

```
<bean id="roleService" class="com.xxxx.service.RoleService" ></bean>
```

Bean对象实例化过程是在Spring容器初始化时被实例化的，但也不是不可改变的，可以通过 lazy-init="true" 属性延迟bean对象的初始化操作，此时再调用getBean 方法时才会进行bean的初始化操作

Bean 的使用

方式一：使用 BeanFactory

```
// 得到Spring的上下文环境  
BeanFactory factory = new ClassPathXmlApplicationContext("spring.xml");  
RoleService roleService = (RoleService) factory.getBean("roleService");
```

方式二：使用 ApplicationContext

```
// 得到Spring的上下文环境  
ApplicationContext ac = new ClassPathXmlApplicationContext("spring.xml");  
RoleService roleService = (RoleService) ac.getBean("roleService");
```

Bean的销毁

实现销毁方式(Spring容器会维护bean对象的管理，可以指定bean对象的销毁所要执行的方法)。

步骤一：实现销毁方式(Spring容器会维护bean对象的管理，可以指定bean对象的销毁所要执行的方法)

```
<bean id="roleService" class="com.xxxx.service.RoleService" destroy-  
method="destroy"></bean>
```

****步骤二：**通过 `AbstractApplicationContext` 对象，调用其`close`方法实现bean的销毁过程

```
AbstractApplicationContext ctx=new ClassPathXmlApplicationContext("spring.xml");  
ctx.close();
```

IOC/DI-控制反转和依赖注入

将对象实例化的创建过程转交给外部容器（IOC容器 充当工厂角色）去负责；属性赋值的操作；